TRI CLOUD ESSENTIALS — AWS

- **Topic:** AWS I — Identity and Storage
- **Date:** 29th October 2026

1. AWS IAM: Users, Roles, Policies, and Trust Relationships

AWS (Amazon Web Services) is Amazon's cloud platform, a large collection of services for computing, storage, databases, and security that run in data centers around the world. Everything in AWS lives inside an AWS account, which is the container for your resources and also the boundary for billing and access.

In simple words. IAM (Identity and Access Management) is the service that decides who can do what on which AWS resource. It is global, so it is not tied to one Region, and there is no charge for using it.

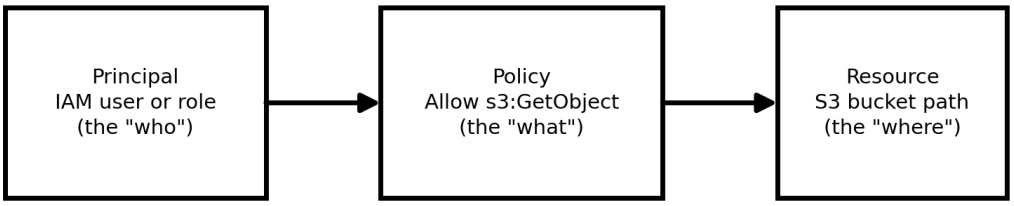
Piece	Simple meaning
Root user	The identity created together with the account, using the sign-up email. It has unlimited power, so protect it with MFA, keep its password safe, and do not use it for daily work.
IAM user	A long-term identity for one person or one application, with a name and credentials such as a console password or access keys. For people, AWS now prefers signing in through IAM Identity Center, which gives temporary credentials.
IAM group	A set of users that share the same permissions. A policy attached to the group applies to every user in it. A group cannot contain another group.
IAM role	An identity with permissions but with no password and no access keys of its own. Someone or something assumes the role and receives temporary credentials.
Policy	A JSON document that lists which actions are allowed or denied, on which resources, and under which conditions.
Access keys	A long-lived pair of an access key ID and a secret access key, used by code to call AWS. They are easy to leak, so prefer roles.

Policies. Permissions are never given directly. They are written in a policy and the policy is attached to an identity or to a resource.

Policy type	Simple meaning

Identity-based policy	Attached to a user, group, or role. It says what that identity may do.
Resource-based policy	Attached to a resource, such as an S3 bucket policy. It names the principal who may use the resource, so it contains a Principal element.
Permissions boundary	A limit set on a user or role. The identity can do only what both its own policies and the boundary allow.
Service control policy (SCP)	A guardrail set in AWS Organizations for whole accounts. It limits what the accounts can do, but it grants nothing by itself.
Element	**What it says**
Version	The policy language version. It is always "2012-10-17".
Statement	The list of rules. Each rule is one block inside the policy.
Effect	Either Allow or Deny.
Action	The API call, written as service:Operation, such as s3:GetObject.
Resource	The thing the action is on, written as an ARN, such as arn:aws:s3:::tricloud-datalake-dev-001/sales/*.
Principal	Who the rule is for. It appears only in resource-based policies and trust policies.
Condition	An extra test, such as the request coming from a given IP address or being signed in with MFA.

An IAM request: who, what, on which resource



Everything is denied by default, and an explicit Deny always beats an Allow

Diagram 1: An identity, a policy, and a resource together decide whether a request is allowed.

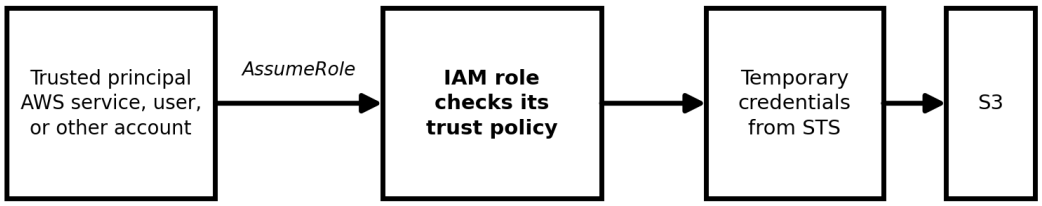
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSalesFiles",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::tricloud-datalake-dev-001/sales/*"
    }
  ]
}
```

How AWS decides. Every request starts as denied. An Allow in a matching policy opens it, and an explicit Deny anywhere closes it again, whatever else says Allow. Inside one account, an Allow in either the identity-based policy or the resource-based policy is enough. Between two accounts, both sides must allow.

Policy kind	Simple meaning
AWS managed policy	Written and updated by AWS, such as AmazonS3ReadOnlyAccess. It is quick to use but often broader than needed.
Customer managed policy	Written by you and reusable. It can be attached to many identities and has versions you can roll back.
Inline policy	Written directly inside one user, group, or role and deleted with it. It is used for a one-off rule.

Roles and trust relationships. A role has two separate policies. The trust policy (also called the trust relationship) says who is allowed to assume the role. The permissions policy says what the role can do after it is assumed. When someone assumes the role, the AWS Security Token Service (STS) hands out temporary credentials: an access key, a secret key, and a session token. They expire, one hour by default, so there is nothing permanent to leak.

How a role is assumed



The trust policy says who may assume the role. The permissions policy says what the role can do.

Diagram 2: The trust policy lets a principal assume the role and receive temporary credentials.

Who can be trusted	**Example**
An AWS service	ec2.amazonaws.com, lambda.amazonaws.com, or glue.amazonaws.com. This is how a service gets its own permissions.
A user or role in the same account	A developer's role that is allowed to switch into an admin role for a task.
Another AWS account	A partner or a separate account for production. The Principal names that account's ID.
An outside identity	A user from a company directory or a CI/CD tool, signed in through SAML or OpenID Connect (OIDC) federation.
Step	**What to do**
1. Open Roles	In the IAM console choose Roles, then Create role.
2. Choose the trusted entity	Pick AWS service, for example Glue, EC2, or Lambda. This writes the trust policy for you.
3. Attach permissions	Attach a managed policy, or a customer managed policy that you wrote for this job.
4. Name and create	Use a clear name such as tricloud-etl-role and add tags.
5. Check it	Use the policy simulator or the Last accessed tab to confirm what the role really uses.

2. The Principle of Least Privilege

In simple words. Least privilege means giving an identity only the permissions it needs for its job, on only the resources it needs, and only for as long as it needs them. If something goes wrong, such as a leaked key or a mistake in a script, the damage stays small.

Dimension	What to limit
Actions	Allow the exact API calls, such as s3:GetObject, and not s3:* or *.
Resources	Name the exact ARN or prefix, such as the sales/ prefix of one bucket, and not every bucket.
Conditions	Add tests such as MFA required, a certain source IP range, or an allowed Region.
Time	Use roles with temporary credentials, which expire, and not access keys that live for years.

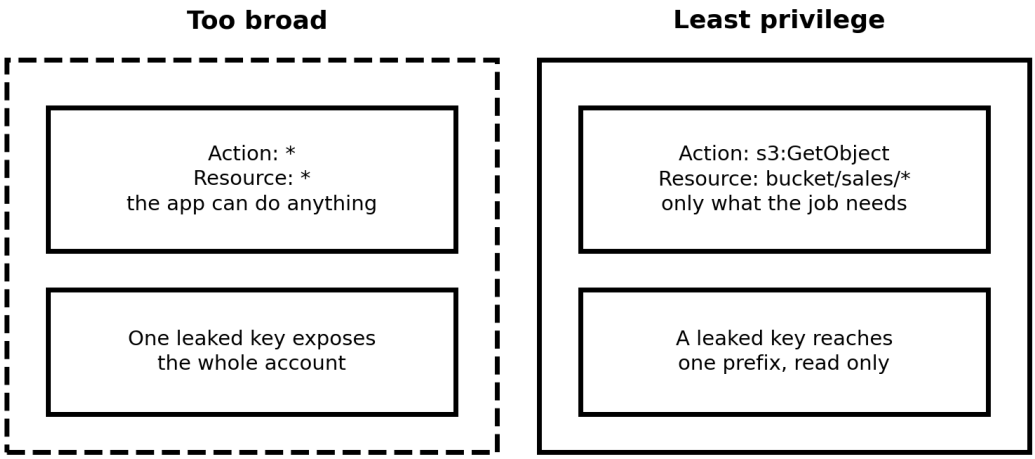


Diagram 3: A broad policy and a least-privilege policy, and what a leaked key can reach in each.

Note. The first rule works on the bucket itself and the second works on the objects inside it. This is why two different ARNs appear. The condition keeps the listing inside the sales/ prefix.

Step	What to do
1. Start small	Begin with no permissions, or with a narrow managed policy, and add only what a real task needs.
2. Write a scoped policy	Replace the broad managed policy with a customer managed policy that names exact actions and ARNs.

3. Let AWS suggest	IAM Access Analyzer can build a policy from the activity recorded in CloudTrail, so the policy matches what was actually used.
4. Remove what is unused	Check the last accessed information of each user and role, and delete the permissions and the access keys that are not used.
5. Add guardrails	Use permissions boundaries and SCPs so that no one can grant more than the organization allows.
6. Review often	Permissions grow over time, so review them on a schedule, for example every quarter.

AWS I & II – Storage, Security, and Pipeline Services

- **Topic:** AWS I & II – Storage, Security, and Pipeline Services
- **Date:** 30th October 2026

3. Amazon S3: Buckets, Objects, and Prefixes

Amazon S3 (Simple Storage Service) is object storage. You store any file as an object inside a bucket and reach it through a web API, a console, or the command line. It is used for data lakes, backups, logs, website files, and the input and output of many other AWS services. S3 is designed for very high durability, 99.999999999% (eleven nines).

Term	**Simple meaning**
Bucket	The top-level container for objects. The name must be unique across all of AWS, and the bucket is created in one Region, where the data stays unless you copy it.
Object	One stored item: the data itself plus its metadata. An object can be up to 5 TB.
Key	The full name of an object inside its bucket, such as sales/2026/10/orders.csv.
Prefix	The leading part of a key that groups objects, such as sales/2026/10/. It works like a folder in the console, but it is only text.
Version ID	With versioning on, each change to an object gets a version ID, so older versions can be recovered.
Storage class	How the object is stored and billed, from S3 Standard for frequent use to the Glacier classes for archives.

One bucket, with prefixes that look like folders

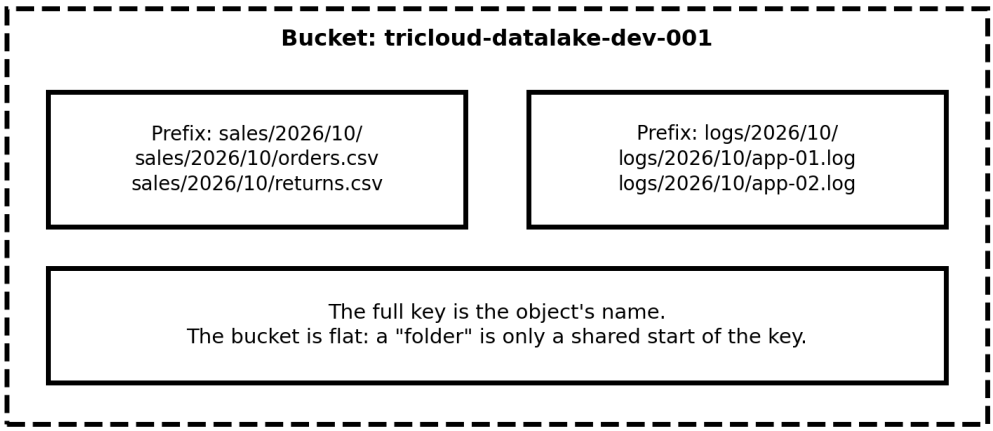


Diagram 4: One flat bucket holds objects, and prefixes only group them by the start of their keys.

Prefixes are not folders. S3 has no real directories. The console shows folders because it splits keys at the "/" character. This means that "renaming a folder" is really copying every object to a new key and deleting the old ones.

Point	**ADLS Gen2 (Azure) vs. S3 (AWS)**
Top container	ADLS Gen2: a container inside a storage account. S3: the bucket, which has a globally unique name.
Folders	ADLS Gen2: real directories, when the hierarchical namespace is on. S3: only prefixes inside the keys.
Rename or move	ADLS Gen2: one fast operation on the directory. S3: copy each object and then delete the original.
Access control	ADLS Gen2: Azure RBAC roles and ACLs. S3: IAM policies, bucket policies, and Block Public Access.
Storage class	**Best for**
S3 Standard	Data used often, such as the active tables of a data lake.
S3 Intelligent-Tiering	Data with unknown or changing use. S3 moves it between tiers automatically.
S3 Standard-IA and One Zone-IA	Data used rarely but needed quickly. One Zone-IA keeps a single copy in one availability zone.

S3 Glacier classes	Archives. Retrieval takes from minutes to hours, and Deep Archive is the cheapest.

Controlling access. S3 can be reached only by identities that are allowed. Several tools work together.

Tool	Simple meaning
IAM policy	Attached to a user or role. It says which buckets and prefixes the identity may use.
Bucket policy	A resource-based policy on the bucket. It names who may use it, including roles from other accounts.
Block Public Access	A switch that stops any public access to the bucket. It is on by default for new buckets, and it should stay on.
ACLs	The older way of setting permissions on objects. New buckets have them turned off, and policies are used instead.
Pre-signed URL	A temporary link, made by someone who has access, that lets another person download or upload one object for a limited time.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEtlRoleRead",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/tricloud-etl-role" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::tricloud-datalake-dev-001/sales/*"
    }
  ]
}
```

Step	What to do
1. Create the bucket	In the S3 console choose Create bucket, enter a unique name such as tricloud-datalake-dev-001, and pick the Region.
2. Keep it private	Leave Block Public Access turned on.
3. Turn on versioning	Enable it if overwritten or deleted data must be recoverable.
4. Check encryption	New objects are encrypted by default. Keep the default, or choose a KMS key (section 4).

5. Create and test	Upload a file, then list the prefix to see it.

4. AWS Secrets Manager and KMS Fundamentals

In simple words. AWS KMS (Key Management Service) creates and guards the cryptographic keys that protect data. AWS Secrets Manager stores secrets, such as database passwords and API keys, and hands them to applications that are allowed to read them. Secrets Manager uses KMS to keep each secret encrypted.

KMS term	**Simple meaning**
KMS key	The key used to encrypt and decrypt. It is created inside KMS, and the key material never leaves KMS in plain form. The default type is a symmetric key.
Alias	A friendly name for a key, such as alias/tricloud-data-key, so that code does not depend on the long key ID.
Key policy	The policy attached to the key. Every key has one, and it controls who may use and manage the key. IAM policies work for a key only if its key policy allows it.
Data key	A small key that KMS creates to encrypt the actual data. It is stored next to the data in an encrypted form.
Envelope encryption	KMS encrypts the data key, and the data key encrypts the data. This way big files are encrypted quickly without sending them to KMS.
Key rotation	KMS can replace the key material every year automatically, with no change needed in the applications.
Who owns the key	**Simple meaning**
AWS owned key	Used by AWS inside its own services. It is free, and it is not visible in your account.
AWS managed key	Created by AWS in your account for a service, such as aws/s3 or aws/secretsmanager. You can see it, but you cannot change its policy.
Customer managed key	Created and controlled by you. You write its policy, choose the rotation, and can disable it. It is billed per month and per request.

Secrets Manager. A secret is a value, often a small JSON document, that a person or application needs to sign in somewhere. Secrets Manager stores it encrypted, and an application reads it with an API call at the moment it is needed, so the password never sits in code or in a file.

Reading a secret that is protected by a KMS key



The role needs permission for the secret, and permission to use the key

Diagram 5: An application with a role asks Secrets Manager, which uses a KMS key to decrypt the secret.

Feature	**Simple meaning**
Automatic rotation	Secrets Manager can change the secret on a schedule using a Lambda function. It has ready-made support for databases such as Amazon RDS.
Versions and labels	A secret keeps versions marked AWSCURRENT, AWSPREVIOUS, and AWSPENDING, so a rotation can happen with no break for applications.
Access control	An IAM policy with secretsmanager:GetSecretValue, and optionally a resource policy on the secret.
Audit	Every read and change is recorded in AWS CloudTrail.
Replication	A secret can be copied to other Regions for disaster recovery.
Point	**Secrets Manager vs. KMS**
What it holds	Secrets Manager: secret values such as passwords and API keys. KMS: cryptographic keys that encrypt and decrypt, whose material never leaves KMS.
Main calls	Secrets Manager: GetSecretValue and PutSecretValue. KMS: Encrypt, Decrypt, and GenerateDataKey.

Rotation	Secrets Manager: rotates the secret value on a schedule. KMS: rotates the key material, once a year when it is turned on.
Azure equivalent	Secrets Manager: Key Vault secrets. KMS: Key Vault keys. Certificates go to AWS Certificate Manager.

S3 encryption	**Simple meaning**
SSE-S3	S3 manages the keys. It is the default for every new object, and it costs nothing extra.
SSE-KMS	A KMS key encrypts the objects, so key use is logged and can be controlled. Readers need kms:Decrypt as well as s3:GetObject.
SSE-C	The caller supplies the key with every request. AWS does not store it.

SESSION 2 — AWS II: PIPELINE BUILDING AND MONITORING

- **Topic:** AWS II — Pipeline Building and Monitoring
- **Date:** 2nd November 2026

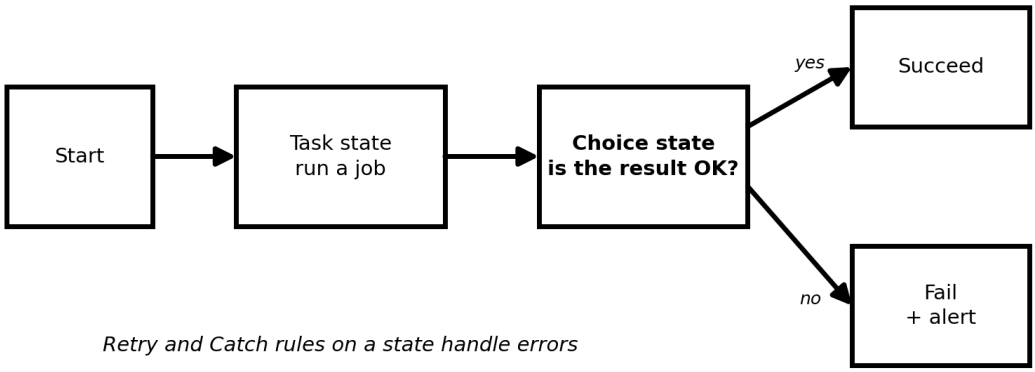
5. AWS Step Functions: Fundamentals and Workflow Orchestration

In simple words. A data pipeline is rarely one task. It is a chain of steps: start something, wait for it, check the result, and do something else if it failed. AWS Step Functions is the serverless service that runs such a chain for you. You describe the steps once, and Step Functions runs them in order, keeps track of where each run is, retries what failed, and records everything. This job is called workflow orchestration.

Term	**Simple meaning**
State machine	The workflow itself, written as a JSON document in the Amazon States Language (ASL). It lists the states and how they connect.
State	One step in the state machine, such as running a job, making a choice, or waiting.
Execution	One run of a state machine. Each execution has its own input, output, status, and history.

Workflow Studio	The visual editor in the console. It lets you drag states onto a canvas and writes the JSON for you.
Execution role	The IAM role that the state machine uses to call other services. Its trust policy names states.amazonaws.com (section 1).
State type	**What it does**
Task	Does the real work by calling a service, such as starting a Glue job, running a Lambda function, or sending an SNS message.
Choice	Looks at the data and picks the next state, like an if and else.
Parallel	Runs several branches at the same time and waits for all of them.
Map	Repeats the same steps for every item in a list, such as each table or each file.
Wait	Pauses for a set time or until a given moment.
Pass	Passes its input on, optionally changing it. It is useful for testing.
Succeed and Fail	End the execution as a success or as a failure, with an error name and a cause.

A state machine: states, a choice, and two endings



Retry and Catch rules on a state handle errors

Diagram 6: A state machine moves from state to state, and each Task can have retry and catch rules.

Waiting for a job. A Task state can call a service in three ways, and the ending of its Resource name tells which one.

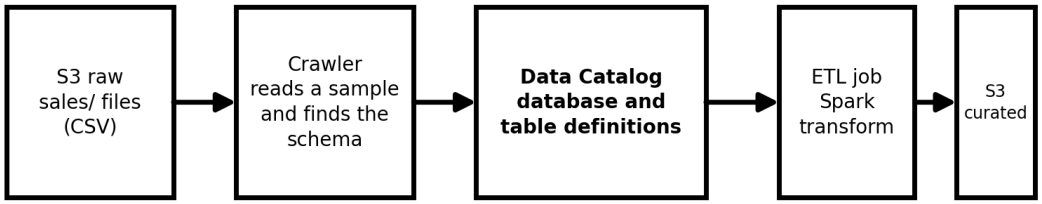
Pattern	Simple meaning
Request and response (default)	Step Functions calls the service and moves on as soon as the call is accepted. It does not wait for the work to finish.
Run a job (.sync)	Step Functions starts the job and waits until it has really finished, then moves on. Glue, ECS, and Batch work this way.
Wait for callback (.waitForTaskToken)	Step Functions hands a token to another system and pauses until that system sends the token back. It is used for approvals and slow outside systems.
Workflow type	**Simple meaning**
Standard	Runs for up to a year, runs each step exactly once, and keeps the execution history for 90 days. It is billed per state transition. This is the choice for data pipelines.
Express	Runs for up to five minutes at a very high rate, and is billed by the number of executions and their duration. History goes to CloudWatch Logs. It suits short event processing.
Way to start	**Simple meaning**
Console or API	A person or program calls StartExecution and passes a JSON input.
EventBridge schedule	A rule or a scheduler starts the state machine at set times, such as every day at 02:00. It plays the role of a trigger.
EventBridge event	An event, such as a new file in S3, starts the state machine.
Another service	Services such as API Gateway or another state machine can start an execution.

6. AWS Glue: Crawlers, Data Catalog, and ETL Jobs

AWS Glue is the serverless data integration service of AWS. It finds out what data you have, records it in a catalog, and runs the code that cleans and moves it. There are no clusters to create. Glue runs the work on managed Apache Spark, and you pay only while the work runs.

Component	Simple meaning
Data Catalog	The central list of metadata: databases and tables that say where the data is, which columns it has, and in which format. It holds the description of the data, and the data itself stays in S3.
Crawler	A program that looks at a data store, reads a sample of the data, works out the schema, and creates or updates the table in the Data Catalog.
Classifier	The rule a crawler uses to recognize a format, such as CSV, JSON, or Parquet. Built-in ones cover the common formats.
Connection	The saved details for reaching a data store such as a JDBC database. The password can be kept in Secrets Manager (section 4).
ETL job	The code that extracts data, transforms it, and loads it somewhere else. It is written in PySpark or Scala, or built in the visual editor.
Glue Studio	The visual editor and notebook environment where jobs are drawn or written and then tested.

AWS Glue: from raw files to a clean table



The catalog stores the schema and the location of the data, and not the data itself

Diagram 7: A crawler records the schema in the Data Catalog, and the ETL job uses that table.

The Data Catalog. A Glue database is only a named group of tables, and a table is a definition: the S3 location, the format, the columns, and the partitions. Services such as Amazon Athena, Redshift Spectrum, EMR, and Glue jobs all read the same catalog, so the schema is described once and used everywhere. Databricks can also use it as a metastore. Access to databases and tables can be controlled in more detail with AWS Lake Formation.

Crawler setting	Simple meaning
Data source	Where to look, such as the S3 path s3://tricloud-datalake-dev-001/raw/sales/, a JDBC database, or DynamoDB.
IAM role	The role the crawler uses. It needs read access to the data and the Glue service permissions, so give it only the prefixes it needs.
Target database	The Glue database that will hold the tables the crawler makes.
Schedule	On demand, hourly, daily, or a custom cron. A crawler can also be started from a pipeline.
Schema change policy	What to do when the data changes: update the table, or only log the change. It also decides what happens to tables whose data was removed.

Partitions. If files are stored in folders such as raw/sales/year=2026/month=10/, the crawler turns year and month into partition columns. Queries that filter on them read only the matching folders, which is faster and cheaper.

Job type	Simple meaning
Spark	The main type. It runs PySpark or Scala code on managed Spark for large data.
Spark Streaming	Reads from a stream, such as Kinesis or Kafka, and processes it continuously.
Python shell	Runs plain Python on a small machine. It suits small tasks such as moving a small file or calling an API.
Job setting	**Simple meaning**
IAM role	The role the job runs with. It reads the input, writes the output, and may read a secret, so it needs those permissions and no more.
DPU and worker type	The size of the job. A DPU (Data Processing Unit) is a measure of compute power. Choose a worker type and a number of workers.
Job bookmark	A memory of what the job has already processed, so that the next run takes only new data.
Job parameters	Values given when the job starts, written as --name value, and read in the script with getResolvedOptions.

Glue version	The version of Spark and Python that the job uses.

Step	What to do
1. Create the role	Create an IAM role that trusts glue.amazonaws.com. Attach the AWSGlueServiceRole managed policy and a scoped S3 policy like the one in section 2.
2. Create a database	In the Glue console create a database such as tricloud_raw.
3. Create the crawler	Point it at s3://tricloud-datalake-dev-001/raw/sales/, give it the role and the database, and choose a schedule.
4. Run the crawler	After the run, a table named sales appears in the database, with its columns and partitions.
5. Create the job	In Glue Studio, read the catalog table, add the transform, and write Parquet to the curated prefix.
6. Run and check	Run the job, then open the job run to see its status, its duration, and its logs in CloudWatch.

7. Step Functions and Glue Together in a Data Pipeline

Glue does the data work, and Step Functions decides the order and handles the problems. Step Functions starts the crawler, waits for it, starts the Glue job with the right parameters, waits for it to finish, and sends an alert if anything fails. Neither service has to know the details of the other.

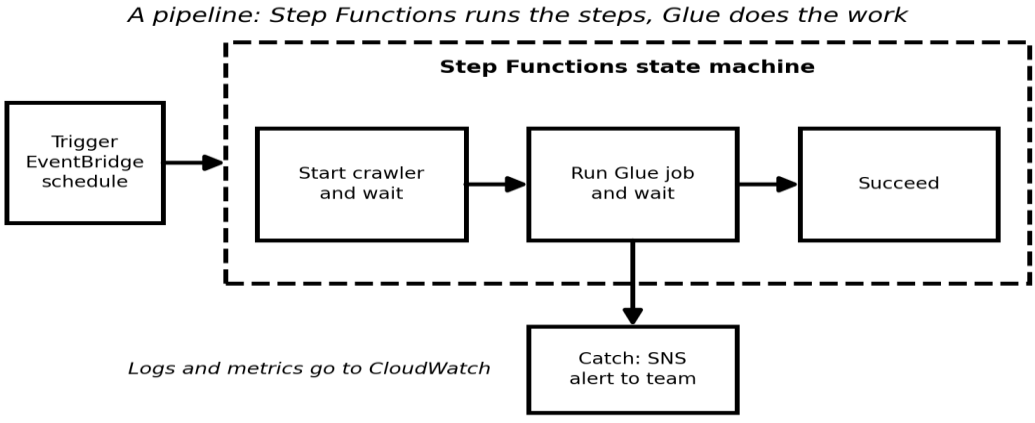


Diagram 8: A schedule starts the state machine, which runs the crawler and the job and alerts on failure.

Step	What happens
1. Start	An EventBridge schedule starts an execution every night and passes the run date as input, such as {"run_date": "2026-11-02"}.
2. Crawl	A Task state starts the crawler. There is no ready-made wait for crawlers, so the state machine loops: Wait, then GetCrawler, then a Choice that checks whether the state is READY.
3. Run the job	A Task state with glue:startJobRun.sync starts the job, passes --run_date, and waits until the run is finished.
4. Check	A Choice or the job result decides whether to continue, for example to a second crawler for the curated table.
5. Alert	A Catch rule sends any error to an SNS topic, and the execution ends as Fail so that it shows as failed.

SESSION 3 — AWS II: PIPELINE SERVICES, MONITORING, AND DATABRICKS CONNECTIVITY

- **Topic:** AWS II — Pipeline Services, Monitoring, and Databricks Connectivity
- **Date:** 3rd November 2026

9. Amazon CloudWatch: Logs, Metrics, and Alarms

Amazon CloudWatch is the monitoring service of AWS. It collects numbers (metrics) and text records (logs) from your resources, shows them in charts and dashboards, and raises an alarm when something looks wrong. For the pipeline of the last session, it answers questions such as "did the Glue job fail?" and "how many executions of the state machine failed today?".

Metric term	Simple meaning
Metric	A series of numbers over time, such as the CPU use of a server or the failed executions of a state machine. Most AWS services publish metrics for you with no setup.
Namespace	The group a metric belongs to, such as AWS/States for Step Functions, AWS/Lambda, or AWS/S3.
Dimension	A name and value pair that picks out one resource, such as StateMachineArn with the ARN of one state machine.

Statistic and period	How data points are combined and over what time. For example, Sum over 5 minutes adds up all failures in each five-minute window.
Retention	One-minute data points are kept for 15 days, five-minute points for 63 days, and one-hour points for 455 days.
Log term	**Simple meaning**
Log event	One record of text with a time stamp, such as one line written by a Glue job.
Log stream	The sequence of log events from one source, such as one job run or one Lambda container.
Log group	A collection of log streams that share the same retention and access rules, such as /aws-glue/jobs/error.
Retention setting	How long the events are kept, from one day up to ten years. The default is to keep them forever, and storage is billed.
Logs Insights	The query tool for logs. You write a short query to search, filter, and count events across one or more log groups.
Metric filter	A rule that counts a pattern in a log group, such as the word ERROR, and publishes the count as a metric that an alarm can watch.

Amazon CloudWatch: from signal to alarm to notification

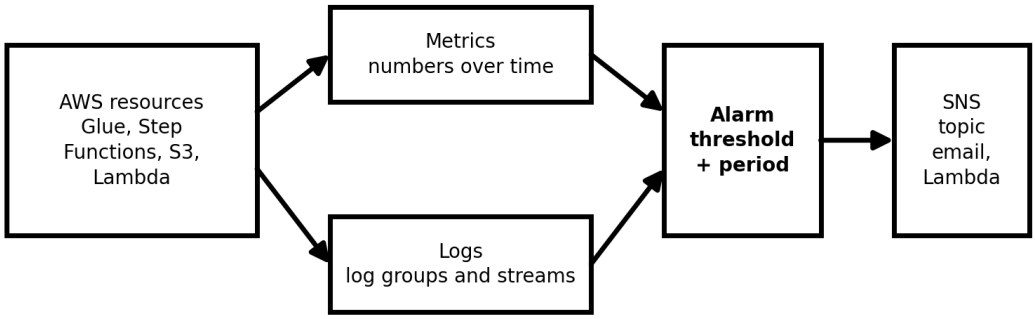


Diagram 9: Resources send metrics and logs to CloudWatch, alarms watch them, and SNS tells people.

Point	Metrics vs. logs
What it holds	Metrics: a number with a time stamp. Logs: a record with text, which can have many details.
Best for	Metrics: charts and quick alarms, such as failed executions above zero. Logs: finding the reason, such as the error message and the line of code.
How to read	Metrics: the Metrics page, graphs, and dashboards. Logs: log groups, live tail, and Logs Insights queries.
Setup	Metrics: most services publish them automatically. Logs: some services write logs only when you turn logging on, and EC2 needs the CloudWatch agent.

Alarms. An alarm watches one metric, or a calculation on several, and changes its state when a rule is broken. It has three states: OK, ALARM, and INSUFFICIENT_DATA.

Alarm setting	Simple meaning
Metric and statistic	What to watch and how to combine it, such as ExecutionsFailed with the Sum statistic.
Period	The length of each window, such as 300 seconds.
Threshold	The limit and the comparison, such as greater than or equal to 1.
Evaluation periods	How many windows must break the rule before the alarm fires, such as 1 out of 1, or 3 out of 5, so that a single spike does not wake anyone.
Missing data	What to do when no data arrives: treat it as missing, as breaching, as not breaching, or ignore it. A job that is silent can be a problem, so choose on purpose.
Actions	What happens on a change of state: publish to an SNS topic, which can send email, SMS, or start a Lambda function, or run an Auto Scaling or Systems Manager action.
Step	**What to do**
1. Create the topic	Create an SNS topic such as tricloud-pipeline-alerts and subscribe an email address. Confirm the subscription from the email.

2. Open Alarms	In the CloudWatch console choose Alarms, then Create alarm, then Select metric.
3. Pick the metric	Choose AWS/States, then State Machine Metrics, then ExecutionsFailed for the state machine of the pipeline.
4. Set the rule	Statistic Sum, period 5 minutes, and the condition greater than or equal to 1.
5. Set the action	When the state is ALARM, send a message to the SNS topic.
6. Test	Run a failing execution and check that the email arrives and the state changes to ALARM.

Point	**CloudWatch vs. CloudTrail vs. EventBridge**
Question it answers	CloudWatch: how is it performing, and is something wrong? CloudTrail: who called which API, and when? EventBridge: something happened, so what should start?
Typical example	CloudWatch: an alarm on failed executions. CloudTrail: who deleted the bucket policy. EventBridge: a rule that starts the state machine every night.

10. Databricks Authentication to Amazon S3

In simple words. Databricks runs its compute in AWS, and its code needs permission to read and write the S3 buckets of the data lake. A person should never paste an access key into a notebook. Databricks instead uses IAM roles, which give temporary credentials, in one of two main ways: an instance profile attached to a cluster, or a Unity Catalog storage credential that governs access by path.

Way to reach S3	**Simple meaning**
Access keys (avoid)	A long-lived key pair stored in a secret scope or in code. It can leak and it does not expire. It is the old way and should not be used.
Instance profile	An IAM role attached to the cluster's EC2 instances. Code on the cluster receives temporary credentials from the instance. It is simple, but every user of the cluster gets the same access.

Unity Catalog storage credential	An object that holds an IAM role. Unity Catalog assumes the role when a user with the right privilege reads or writes a path. Access is granted per group and per path. This is the recommended way.
IAM role	**What it is for**
Cross-account role	Created in your AWS account so that the Databricks control plane can create and manage clusters and networking in your account. It is about running the workspace and not about your data.
Instance profile role	Assumed by the EC2 instances of a cluster. It carries the S3 permissions for that cluster. Its trust policy names ec2.amazonaws.com (section 1).
Storage credential role	Assumed by Unity Catalog for governed S3 access. Its trust policy names the Unity Catalog role and the role itself, with an external ID.

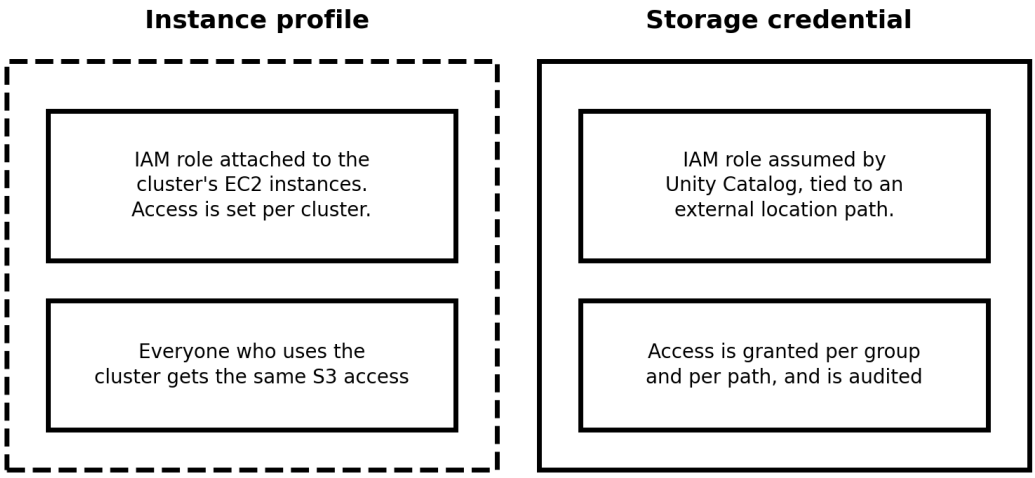


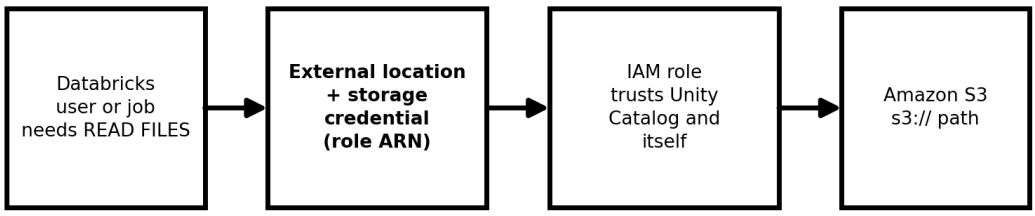
Diagram 10: An instance profile is set per cluster, and a storage credential is governed per path.

Instance profile. Use it for older clusters that are not governed by Unity Catalog. It has two IAM pieces: the role, with a trust policy for EC2 and an S3 permissions policy, and the instance profile, which is the container that lets EC2 use the role.

Step	What to do
1. Create the role	Create an IAM role for EC2 that trusts ec2.amazonaws.com, and attach a scoped S3 policy, as in section 2.
2. Create the instance profile	The console makes one with the same name when you create the role. With the CLI, create it and add the role to it.
3. Allow the pass	Add iam:PassRole for the role to the cross-account role of Databricks, so Databricks may launch clusters that use it.
4. Register it	In the Databricks admin settings, add the instance profile ARN.
5. Choose who may use it	Give only the people or groups that need it the permission to use the instance profile.
6. Use it on a cluster	Select the instance profile in the cluster settings, start the cluster, and test a read.

Unity Catalog storage credential. Here the role is not attached to a cluster. Unity Catalog assumes it only when a user who holds the privilege asks for a path. The role must trust Unity Catalog and, because of how Databricks assumes it, must also trust itself.

How Unity Catalog reaches S3 with a storage credential



Unity Catalog checks the privilege, then assumes the role and receives temporary credentials

Diagram 11: Unity Catalog checks the privilege, assumes the IAM role, and reads S3 with temporary credentials.

Step	What to do
1. Create the role	Create an IAM role for the storage credential, such as tricloud-databricks-s3-role. A first, temporary trust policy is enough.
2. Create the credential	In Databricks open Catalog, then External data, then Credentials, and create a storage credential with the ARN of the role. Databricks then shows the values for the trust policy, including an external ID.
3. Set the trust policy	Update the role so that it trusts the Unity Catalog role from Databricks and the role itself, with the external ID as a condition.
4. Attach the S3 policy	Allow the S3 actions on the bucket or prefix, and let the role assume itself.
5. Create the external location	Create an external location with the s3:// path and the storage credential.
6. Grant and test	Grant READ FILES or WRITE FILES to a group, and list the path to confirm it works.

Point	Azure vs. AWS for Databricks and storage
Identity object	Azure: the Access Connector, which holds a managed identity. AWS: an IAM role that Unity Catalog assumes.
Permission on storage	Azure: a role such as Storage Blob Data Contributor on the storage account. AWS: an S3 policy attached to the IAM role.
Unity Catalog objects	Azure: storage credential, then external location with abfss://. AWS: storage credential, then external location with s3://.
Cluster-level option	Azure: none. The Access Connector is used instead. AWS: instance profile, an IAM role attached to the cluster.